TensorGuard: An Auditable Static Safety Net for PyTorch Tensor Programs

halley-labs

Generated from repository artifacts, June 2026

Abstract

TensorGuard is a zero-annotation static verifier for PyTorch `nn.Module` architectures and adjacent training/deployment code. It checks tensor shape, dtype, device, phase, gradient-flow, stride/layout, probabilistic batch/event contracts, complex view/FFT shape-dtype rules, named-axis alignment, `vmap` batched-dimension transfers, sparse layout/blocksize invariants, packed/unpacked MultiheadAttention q/k/v and mask contracts, geometric sampler grids, and library-specific contracts before a forward pass runs, then reports a three-valued verdict: Safe, Unknown, or Unsafe. The implementation is intentionally more than a prototype: it ships a formal soundness contract, a verifiable-fragment specification, operator confidence tags, SARIF/JSON/JUnit/GitHub reporters, pre-commit and pytest integrations, a safe AST/fx-only loading path for untrusted model files, a community stub governance process, and an upstream PyTorch hook proposal.

The evidence base is also first-class. A frozen real benchmark corpus, a 2,704-item public GitHub bug snapshot, a 227-case generated corpus, a blind held-out split, fuzzers, mutation tests, live PyTorch dispatcher comparisons, statistical tests, numeric-claim audits, and a one-command reproducibility capsule all live in the repository. On the frozen 16-case real benchmark corpus, TensorGuard reports TP=8, FP=0, TN=8, FN=0; on 80 clean sound-mode models and 200 random clean fuzzed modules it reports zero false positives; on 281 genuine injected faults it reports perfect recall; and on 2,000 dispatcher-generated modules it agrees exactly with eager PyTorch with no abstentions. The formal side includes a Lean 4 library whose 371 public theorems are discovered and axiom-audited by tests to depend only on the trusted Lean kernel axioms and not on `sorryAx`, plus parity checks against the Python verifier, Z3, cvc5, and the live torch dispatcher.

Contents

1 Why this tool exists

Tensor bugs are unusually expensive because a model can be syntactically valid, type-check in Python, instantiate successfully, and still crash only after a particular batch shape, device placement, train/eval mode, attention mask, or export path reaches the wrong operator. Runtime tests catch some of these bugs, but only when the test constructs the right input and executes the failing path. Smoke tests miss latent eval-only branches, CUDA-only device mismatches on CPU machines, silent gradient breaks, and shape constraints hidden inside library calls such as `einops.rearrange`, `scaled_dot_product_attention`, `grid_sample`, or PyTorch named-tensor `align_to`.

TensorGuard attacks that failure mode with static verification. Given model source or a live module, it extracts a symbolic tensor program, propagates abstract tensor facts through operator transfer functions, emits solver obligations, and surfaces either a proof of safety within the declared fragment, a concrete refutation, or an explicit abstention. The design choice that pervades the repository is honesty: when the tool cannot soundly prove a program safe, the strict mode returns Unknown rather than a silent pass; when a benchmark number depends on CUDA, HuggingFace downloads, or a Lean toolchain, the numeric audit labels it environment-qualified instead of pretending it was regenerated in the ordinary CPU-only path.

What is new in this repository. TensorGuard combines a production-facing developer tool with a paper-grade evidence machine. The same tree contains the verifier, its soundness contract, its generated specs, its benchmark corpora, the comparison baselines, the statistical tests, the dashboard, the artifact-evaluation capsule, and the formal Lean audit. This paper summarizes those assets as a coherent system rather than as a sequence of roadmap checkboxes.

Core claims.

1. **No-annotation PyTorch verification.** The common path is `tensorguard verify model.py`; the verifier infers shapes and other tensor facts from constructors, `forward`, dataflow, and optional input specs.

2. **Sound abstention boundary.** Strict `sound` mode returns Safe only inside the verifiable fragment and returns Unknown for unsupported constructs. More permissive modes trade abstention for legacy recall while preserving the reported bug set.

3. **Cross-domain bug coverage.** Shape, device, dtype, phase, and gradient-flow checks are measured separately. Device and gradient each catch real bugs missed by a shape-only view; phase is reported as diagnostic-only where it does not refute.

4. **Repository-anchored evaluation.** Headline numbers are generated from committed artifacts and checked by `reproducibility/audit_numeric_claims.py`. The audit currently reports 12 verified claims, one regime-qualified claim, and one environment-qualified claim.

5. **Machine-checked core obligations.** Lean modules model the central operator rules, reduced products, CEGAR bounds, fragment modes, and SMT encodings; tests discover and axiom-audit all 371 public theorems.

2 User-visible capabilities

The repository is not just a verifier kernel. It exposes an adoption surface that lets users run TensorGuard locally, in CI, in editors, and in upstream-style hooks.

2.1 Command-line and Python API

The CLI entrypoint is `tensorguard`. Its central command verifies a file or module and supports independent check toggles, soundness modes, and machine readable output. The public Python API mirrors the CLI through `verify_architecture` and `verify_module`. Specialized helpers cover einops, SDPA, MultiheadAttention, distributions, named tensors, complex FFT/view contracts, `vmap` batched-dimension transfer, sparse tensor layouts, and `grid_sample`/`affine_grid`. A typical workflow is:

```
import torch.nn as nn
from src.api import verify_architecture

class Bad(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(8, 16)
        self.fc2 = nn.Linear(99, 4)

    def forward(self, x):
        return self.fc2(self.fc1(x))

result = verify_architecture(Bad, input_shapes={"x": (2, 8)},
                             soundness_mode="sound")
assert result.verdict == "UNSAFE"
```

The CLI can emit human-readable diagnostics as well as JSON, SARIF 2.1.0, JUnit-XML, and GitHub annotation formats. SARIF is validated for GitHub Code Scanning. The pytest plugin lets projects fail tests on unsafe modules. The pre-commit hook and GitHub Action paths allow incremental rollout, and baseline or suppression support lets teams adopt the tool without blocking on a legacy backlog.

2.2 Soundness modes and exits

TensorGuard exposes three modes:

Mode	Contract
`sound`	Report Safe only when the analyzed region is fully inside the verifiable fragment, uses no heuristic-tagged operators, and has no opaque layers. Unsupported code returns Unknown; the CLI exits 2 on Unknown so CI can gate on abstentions.
`balanced`	Default mode. Preserves legacy usability by abstaining on opaque layers but otherwise returning Safe when no bug is found.
`heuristic`	Most permissive mode. Tolerates heuristic regions and is useful for exploratory diagnostics, never for a "proof of safe" claim.

The mode affects the top-level verdict, not the set of reported bugs. A real shape bug is refuted identically in all three modes, which prevents users from accidentally hiding an error by choosing a more conservative contract.

2.3 Diagnostics, explanations, and adoption hooks

The verifier produces source-mapped diagnostics with category, confidence, line location, counterexample facts, and a suggested repair where the fix is mechanical. The `--explain` path reconstructs the inference

chain that led to a bug. The `@tensorguard.checked` decorator and the upstream `attach_verifier` reference hook demonstrate an opt-in PyTorch integration: a module is verified once at the first forward boundary, rejected before the kernel runs if unsafe, or allowed transparently if safe.

2.4 Security and safe loading

Model verification often starts from untrusted Python files. The repository therefore contains a threat model, security policy, and a safe AST/fx-only path that avoids dynamic import or `exec` when analyzing source. Dynamic execution remains available only in explicitly-labeled harnesses whose purpose is to compare against real PyTorch behavior. The production verifier treats unsupported or opaque regions as UNKNOWN in sound mode rather than executing them.

3 Program model and contracts

3.1 Verifiable fragment

`src/verifiable_fragment.py` is the executable source of truth for what TENSORGUARD can analyze. It defines a grammar for supported `nn.Module` and `forward` constructs, tables of supported layers, tensor methods, `torch.*` functions, and `torch.nn.functional` calls, plus an unsupported-construct taxonomy. The generated document `VERIFIABLE_FRAGMENT.md` states the policy in user language: unsupported code must be surfaced as UNKNOWN in sound mode, never silently passed.

Static checks catch data-dependent control flow, data-dependent iteration, dynamic assertions, and tensor-to-scalar coercions such as `.item()`. Tests run all statically detectable out-of-fragment categories through the live verifier and confirm they yield UNKNOWN under the sound contract.

3.2 Soundness contract

`src/soundness_contract.py` and `SOUNDNESS_CONTRACT.md` state two directions separately:

Refutation soundness. A reported bug is backed by a concrete violated obligation. The user should not have to distinguish a real bug from a solver artifact.

Verification soundness. A SAFE verdict in sound mode means no modeled bug class is missed within the verifiable fragment and the current operator confidence envelope.

The contract classifies constructs as `sound`, `over_approximated`, `under_approximated`, or `skipped`. Known gaps are named rather than hidden. For example, permissive modes may return SAFE on out-of-fragment code; sound mode abstains. CEGAR's historic SAFE-on-infeasible gap is closed by returning abstention rather than a false SAFE.

3.3 Abstract domains

The analysis tracks a reduced product of tensor facts. The original shape domain contains rank, concrete and symbolic dimensions, broadcast facts, divisibility obligations, stride/layout, and permutation facts. Additional domains track dtype promotion, device placement, train/eval phase, and gradient flow. Reductions exchange facts across domains: a dtype cast into a floating layer matters for shape-level operator validity; a device transfer changes the device obligation at downstream ops; a branch guarded by training mode exposes phase-dependent shape paths.

The domain-ablation artifacts distinguish verification domains from diagnostic domains. Shape, dtype, device, and gradient each earn their place by catching bugs in a labeled corpus; phase currently records train/eval structure and enables phase-specific diagnostics but is reported honestly where it does not independently refute.

3.4 Operator confidence

Every registered operator transfer function has a machine-readable confidence tag in `operator_confidence_table.json`. The current table contains 117 operators: 75 `complete`, 36 `sound`, and 6 `heuristic`. Unknown operators default to `heuristic`. The CLI command `tensorguard operator-confidence` surfaces these tags to users, and sound mode folds heuristic operators into UNKNOWN rather than claiming proof.

4 Extraction frontends

TENSORGUARD offers multiple frontends that lower user code to a shared internal representation.

AST frontend. The safe default parses source without importing the target module. It reads constructor assignments, `forward` dataflow, helper calls, subclass patterns, literal `torch.randn` shapes, and simple input annotations. It is the right path for CI, untrusted files, and repository-wide scans.

`torch.fx` frontend. For live modules, the fx frontend traces idiomatic architectures and lowers operator calls into the same transfer-function registry. The trace-success study covers 21 torchvision models, all traced and lowered, for 5,238 total steps with 7.22% unsupported operations recorded rather than hidden.

`torch.export`/Dynamo frontend. The export frontend reconciles PyTorch's capture path with the fx path. The current reconciliation artifact contains seven representative models, with both frontends correct on all seven and no divergent verdicts. Dynamic-shape export constraints and export failure modes are surfaced through the same verifiable-fragment taxonomy.

Second framework: Flax/JAX. The verifier core is framework-independent enough to support a Flax/JAX frontend for `nnx.Sequential`-style models. Linear, LayerNorm, BatchNorm, and Dropout lower into the shared IR, shape bugs refute, and unsupported layouts abstain soundly. This demonstrates that the project is a tensor-program verifier rather than a PyTorch-only parser trick.

Community stubs and autogeneration. Third-party layers are handled through governed declarative stubs: no executable code, mandatory provenance, and executed conformance cases. A stub autogenerator derives safe shape stubs from real `torch.nn` constructor signatures when the output contract is exactly known, and classifies uncertain layers as unsupported rather than guessing.

5 Constraint solving and refinement

5.1 Transfer functions

Operator transfer functions are the heart of the analyzer. They cover linear layers, convolution and pooling families, normalization layers, embeddings, matmul and batched matmul, broadcasting arithmetic, reshape/view/flatten, transpose/permute, concatenation, split/chunk, gather/scatter, reductions, attention blocks, dtype casts, device moves, train/eval-sensitive ops, and gradient-flow constructs such as `detach`. Frequency-weighted coverage on a 13-model torchvision corpus is 94.86%: 2,437 of 2,569 observed operator occurrences are covered, with the remaining 132 occurrences listed by name.

5.2 Solver obligations

The verifier builds verification conditions over integer shape arithmetic, finite-domain device and phase facts, dtype lattice facts, and gradient-flow bits. Z3 is the default backend because the implementation relies on its incremental solver and user-propagator support. A cvc5 comparison harness checks a curated set of shape, device, phase, and reshape obligations through both solvers and records concordant SAT/UNSAT answers, including on the nonlinear reshape fragment.

The solver-domain flags `--no-device-check`, `--no-phase-check`, and `--no-grad-check` gate constraint generation itself, not merely a post-hoc filter. When a domain is disabled, its encoder emits no constraints and its theory-specific checks are skipped; committed examples prove that each flag can flip a verdict on real source.

5.3 CEGAR predicate discovery

The CEGAR loop discovers implicit shape contracts by accumulating predicates from counterexamples. If predicates discovered across refinement rounds are jointly unsatisfiable, the conflict is promoted to a real bug of category `cegar_refined_contract`. This makes the iteration budget observable: `--cegar-iterations 0` can produce no contract-level bug while a positive budget surfaces the infeasible contract. A convergence study proves the loop terminates within a tight bound, and a Lean proof models the monotone predicate accumulation argument.

5.4 Specialized library contracts

Seven high-value library checkers demonstrate the pattern for precision without unsoundness:

- **einops.** `src/einops_verify.py` models `rearrange`, `reduce`, and `repeat`; it catches non-divisible decompositions, underdetermined splits, axis-set mismatches, duplicate axes, anonymous-axis errors, rank mismatches, and ellipsis cases. A 3,000-case differential fuzz battery checks verdicts and output shapes against the real `einops` package. Symbolic divisibility that cannot be decided statically is not refuted.

- **Scaled dot-product attention.** `src/sdpa_verify.py` models the rank, head-dimension, batch/-head broadcasting, and mask broadcasting contract of `F.scaled_dot_product_attention`. A 400-case differential battery checks it against real PyTorch. Backend-specific key/value sequence-length behavior is not hard-refuted where PyTorch's default fused backend accepts it.

- **`torch.distributions`.** `src/distributions_verify.py` resolves batch shapes, event shapes, and `log_prob` output shapes for scalar-event distributions, `Categorical`, `MultivariateNormal`, and `Independent`. The contract is intentionally over usable `sample`/`log_prob` behavior, not just constructors, because PyTorch can build a malformed multivariate distribution that only fails when used. Differential tests compare against real PyTorch distributions with support validation disabled so the oracle isolates shape failures.

- **Complex views and FFT.** `src/complex_verify.py` models `torch.view_as_real`, `torch.view_as_complex`, and the core `torch.fft` family. It captures PyTorch's concrete shape rules for `n`, `s`, and `dim`, including no-op `fftn(..., dim=())`, Hermitian half-spectrum lengths, inverse real-FFT length recovery, out-of-range and duplicate dimensions, and the exact dtype edge cases: `rfft` rejects complex inputs, `view_as_complex` accepts only half/float/double real views, and FFT kernels reject half, bfloat16, and complex-half where the live dispatcher does. Unknown dtypes and symbolic lengths abstain instead of refuting. `tests/test_complex_verify.py` differentially checks the rules against real PyTorch across view, single-axis FFT, and multi-axis FFT cases.

- **PyTorch named tensors.** `src/named_tensor_verify.py` models `refine_names` and `align_to`: an existing concrete axis name cannot be renamed or demoted, Ellipsis carries omitted axes in torch's order, new target names insert singleton dimensions, and unnamed dimensions are moved only when covered by Ellipsis. A conservative source checker reports invalid literal named-tensor calls in real code. Differential tests compare random refine/alignment cases against real PyTorch named tensors, including duplicate names, invalid identifiers, explicit `None` axes, and symbolic sizes carried through unchanged.

- **`torch.vmap`/functorch.** `src/vmap_verify.py` models the higher-order shape transfer used by `torch.vmap` and `torch.func.vmap`: mapped input axes are removed before the body, concrete mapped batch sizes must agree, and the shared batch axis is reinserted according to `out_dims`. The checker supports nested output pytrees and treats symbolic batch dimensions by abstention. For PyTorch's version-sensitive constant-output corner, it records an unknown shape rather than refuting. A 500-case randomized differential test checks mapped-axis removal and out-dim insertion against real `vmap`.

- **`torch.sparse`.** `src/sparse_verify.py` models COO sparse/dense rank agreement and the validated CSR/CSC/BSR/BSC compressed-layout contract: batch metadata, nnz agreement, compressed-axis lengths, positive block sizes, and block divisibility. It also flags compressed tensors whose values' dense tail disagrees with the requested tensor shape: PyTorch may construct these metadata-inconsistent tensors, but `to_dense()` fails, so TensorGuard reports them as unusable layouts. The tests compare valid and invalid cases against real PyTorch constructors with `check_invariants=True`, plus the live dense-materialization failure and symbolic no-false-positive probes.

- **`grid_sample`/`affine_grid`.** `src/grid_sample_verify.py` models the 2-D and 3-D geometric sampler contracts: input/grid rank, batch equality, coordinate-grid trailing size, non-empty input spatial axes, bicubic's 4-D restriction, dtype agreement, and `affine_grid`'s theta-matrix and positive-size requirements. The tests replay hand edge cases and randomized cases against real PyTorch, including the easy-to-get-wrong distinction that `grid_sample` accepts empty output grids while rejecting empty input spatial axes.

- **`nn.MultiheadAttention`.** `src/mha_verify.py` models packed and separate q/k/v projections, `kdim`/`vdim`, `batch_first`, 2-D and 3-D attention masks, key-padding masks, and attention-weight output shapes. Nested tensors abstain rather than fabricating rectangular dimensions. Differential tests compare module and `multi_head_attention_forward` calls against real PyTorch, and the `torch.fx` module path carries explicit q/k/v/mask roles into the verifier.

6 Beyond `forward`: deployment and training hazards

The repository extends static checking beyond architecture shape.

Training-loop hazards. `src/training_loop_checks.py` analyzes optimizer and gradient-flow patterns: detached losses, missing `zero_grad`, missing `optimizer.step`, `zero_grad` after `backward`, and fp16 autocast without a `GradScaler`. The reproducibility harness runs equivalent seeded PyTorch loops and confirms the static verdict matches the runtime symptom.

Distributed tensor parallelism. `src/tensor_parallel_checks.py` models Megatron-style column-parallel and row-parallel linear stacks. It checks shard divisibility, contracted-dimension agreement, and communication flag consistency. The harness hand-shards a reference stack across simulated ranks and

verifies that correct configs match the unsharded forward while inconsistent configs are both statically flagged and fail under actual sharding.

Quantization and export. `src/quant_export_checks.py` catches asymmetric quant/dequant boundaries, quantized tensor arithmetic that should use `FloatFunctional`, and export-unsafe data-dependent constructs. The live PyTorch harness confirms the quantized-add hazard raises the expected runtime exception and that export failures line up with the verifiable-fragment grammar.

ONNX, `torch.compile`, and packaging gates. The repository includes ONNX hardening, post-export `check_model` assertions, and torch.export/AOTInductor packaging parity. These gates are framed as deployment checks: they do not broaden the soundness contract unless their corresponding transfer rules and runtime conformance tests are present.

7 Formal assurance

7.1 Lean proof corpus

The Lean 4 development under `lean/TensorGuard/` mechanizes the proof obligations that are most dangerous to leave as prose: operator rules, reduced-product algebra, CEGAR bounds, fragment modes, device/dtype/phase/grad transfers, shape-chain transfers, and SMT encoding faithfulness. The whole library audit test reads the root import graph, discovers every public theorem and lemma, generates `#print axioms` commands for each, elaborates them through the Lean kernel, and asserts that no declaration depends on `sorryAx` and that every reported axiom is one of `propext`, `Classical.choice`, or `Quot.sound`. The current discovered count is 371 public theorems.

The proof corpus is paired with Python tests rather than left disconnected. Conformance tests map Lean theorem names to randomized property tests of the Python code. Handler parity checks compare Lean-extracted or Lean-modeled rules with the Python transfer functions and, where applicable, the live torch dispatcher. This is important: a theorem about a toy rule is not enough unless the implementation that users run is pinned to the same semantics.

7.2 Encoding faithfulness

Several Lean modules address the trust boundary between abstract transfer rules and solver encodings. `lean/TensorGuard/SmtEncoding.lean` proves that the generated Z3 obligations for dtype/device/phase/gradient facts faithfully represent the Lean semantics. `lean/TensorGuard/CrossDomain.lean` lifts shape/device interactions. Additional modules cover dtype promotion chains, gradient-flow chains, device-placement chains, phase chains, broadcast chains, rank-transfer chains, flatten, cat, embedding, reshape `-1` inference, Linear, Conv1d/Conv2d, ConvTranspose1d, pooling, LayerNorm, PixelShuffle, AdaptiveAvgPool2d, Unflatten, and BatchNorm.

7.3 Soundness footprint

The paper-grade soundness story is deliberately scoped. The repository distinguishes Lean-audited handlers, pen-and-paper handlers, tested-only handlers, and uncovered handlers. Per-block soundness-footprint artifacts pin real refutations to the concrete handler set they traversed, including the specific Lean theorem or rule that discharges each audited handler. This prevents a misleading global statement such as "Lean proves the whole tool" from masking uncovered long-tail operators.

8 Benchmark corpora

The benchmark strategy uses multiple corpora because no single corpus can answer every reviewer question.

Table 1: Committed corpora and what each one proves.

Corpus / source	Size	Purpose
`real_benchmarks/`	16	Frozen ground-truth real-code corpus: 8 clean and 8 buggy modules across shape, device, phase, and gradient domains, each content-addressed by SHA-256. Six of eight buggy modules raise a real eager PyTorch RuntimeError on this host; the CUDA-only and silent-gradient cases are documented.
`experiments_v5/github_bug_mining/`	2,704	Public GitHub issue/PR snapshot mined from verbatim PyTorch runtime error signatures: 2,394 shape and 310 device bugs across eight categories.
`corpus_extended/`	227	Parameterized, redistributable generated corpus: 153 buggy and 74 clean cases, all labels validated by real PyTorch execution and content-addressed.
Blind held-out split	186	Pre-registered split disjoint from development parameter grids: 138 buggy and 48 clean cases, scored exactly once.
False-positive stress corpus	101	Clean models built from tricky but valid shape, broadcast, reshape, concat, and normalization patterns.
Natural-distribution study	29	Clean public-style architectures across fourteen families; measures definite-answer coverage on ordinary code.
Differential dispatcher corpus	2,000	Random modules judged by both eager PyTorch and TensorGuard sound mode; tests exact agreement with the live dispatcher.
Hypothesis module-AST corpus	800	Structured generated modules with shrinking, testing the full-module soundness property rather than isolated operators.
Mutation corpus	376 genuine mutants	Mutations of validated-clean parents; only mutants that genuinely raise under eager PyTorch are admitted.

Why not only real bugs? The frozen real corpus provides provenance; the GitHub-mined snapshot provides scale; generated corpora provide redistributability and controlled labels; blind splits test overfitting; fuzzing and mutation testing search for regressions in both directions. The combination is what makes the evaluation credible.

9 Main empirical results

9.1 Frozen real-code benchmark

Table 2 reports the committed confusion matrices on the 16-case frozen corpus. Baselines are actual tools or executable procedures: a no-op predictor, a runtime forward smoke test, a runtime forward+backward test that also inspects gradients, and PyTea built from source and run live on generated entry wrappers.

Method	TP	FP	TN	FN	Precision	Recall
TensorGuard	8	0	8	0	1.000	1.000
runtime backward	8	0	8	0	1.000	1.000
runtime forward	7	0	8	1	1.000	0.875
PyTea	6	0	8	2	1.000	0.750
no-op	0	0	8	8	–	0.000

Table 2: Frozen 16-case corpus; the backing artifact is evaluation/confusion_matrices.json.

The runtime backward baseline ties on this small corpus because each bug is exercised by the seeded run and because the gradient-detach case can be caught by checking parameter gradients. That tie is useful: it shows the comparison is not rigged against dynamic tools. The static advantage appears on latent bugs where one concrete execution cannot exercise the failing path.

9.2 Latent bugs

The hard-recall corpus contains eight bugs that are all proven genuine but silent under a single seeded train-mode forward/backward baseline: three eval-only phase bugs, three path-flag bugs, and two silent parameter-freeze bugs. The strongest dynamic baseline catches 0/8. TensorGuard catches 6/8, covering the phase and path families in full; the two misses are explicitly tagged as post-construction `requires_grad=False` freezes outside the current gradient-domain model. This is exactly the failure mode static analysis should improve: not "faster execution", but detection of bugs no chosen execution observes.

9.3 False-positive and false-negative hunts

The most important engineering property is not a single high recall number; it is the absence of easy counterexamples. The project therefore attacks the tool from both sides: generate clean models and demand no false alarms, then inject real faults and demand no misses. When these hunts find no natural disagreements, the repository still freezes a regression suite of minimal buggy/clean siblings so future changes cannot accidentally regress.

9.4 Historical and mined bugs

The historical 60-bug corpus reports two clearly-labeled regimes. In the default headline regime, TensorGuard produces Refuted-Proof verdicts on 53/60 bugs at the high-confidence threshold, with seven silent misses. In the raw-refute regime, which lifts per-bug input shapes and allows CEGAR depth three, it refutes 56/60, 55 of them high-confidence. The audit harness recomputes both numbers from `reproducibility/reproduce_headline_60bug.json`. The larger GitHub-mined snapshot is not a precision/recall benchmark by itself; it is a frozen, content-addressed source of real-world bug signatures and categories used to drive future corpus growth.

9.5 Operator and frontend coverage

Coverage is measured two ways. The registry-level table tags 117 transfer functions by confidence. The real-model frequency study asks how often those rules appear on 13 torchvision architectures: 94.86% of 2,569 operator occurrences are covered. The fx trace-success study asks whether the frontend can capture real models: 21/21 torchvision models trace and lower, totaling 5,238 steps; unsupported operations are counted and reported, not ignored.

Study	Population	Result
Sound-mode clean hunt	80 clean models	0 false positives; 80/80 SAFE; no abstentions.
Differential clean fuzzing	200 random clean models	0 false positives; 200/200 SAFE; ranks split 94 rank-2 and 106 rank-4.
Negative fuzzing	281 genuine injected faults	281/281 caught; 0 false negatives across Linear-in, Linear-out, Conv-in, and bad-reshape families.
Disagreement triage	481 fuzzed clean/-faulty models	0 TensorGuard/runtime disagreements; 50 minimal bug reproducers frozen as regressions.
Extended corpus	227 cases	153/153 runtime-failing cases caught; 0 false positives on 74 clean cases in both balanced and sound modes.
Blind split	186 cases	138/138 held-out buggy cases caught; 0 false positives on 48 clean cases; zero overfitting gap.
Live dispatcher differential	2,000 modules	Exact agreement with eager PyTorch: zero soundness violations, zero false alarms, zero abstentions.
Mutation testing	376 genuine mutants	Sound mode kills every admitted mutant; no genuine bug reported SAFE.

Table 3: Independent robustness studies. Each row is generated by a separate script and pinned by tests.

9.6 Localization and developer effort

For refuted real bugs with author-placed `# BUG` markers, the localization proxy measures how many lines a developer inspects to reach the true bug. Median assisted effort is two lines; median unaided effort is 7.5 lines; the median reduction factor is 3.0 with a bootstrap interval of [1.33125, 5.875]. TENSORGUARD helps on 12 of 14 bugs and hurts on two, which are kept in the artifact. The repository also contains a pre-registered human-study protocol, because this proxy is not claimed to be a human RCT.

9.7 Statistical testing

`evaluation/significance.py` runs paired exact McNemar tests with Holm-Bonferroni correction and paired bootstrap confidence intervals over the confusion-matrix predictions. On the 16-case frozen corpus, TENSORGUARD significantly beats the no-op floor after correction, never loses a discordant pair to any baseline, and does not overclaim significance over PyTea at this sample size. `reproducibility/effect_sizes.py` adds paired effect sizes and both FWER and FDR corrections. `reproducibility/statistical_power.py` reports sample-size justifications and rule-of-three bounds for zero-false-alarm claims.

9.8 Scaling

The scaling study sweeps feed-forward depth from one to 64 layers. The structural-work metric is linear in depth; the wall-clock companion has a log-log scaling exponent of 1.69 and $R^2 = 0.988$, below cubic and far from an exponential blow-up. Median wall time grows from about 12 ms at depth one to about 4.26 s at depth 64 on the recorded host.

10 Reproducibility machinery

10.1 One command for the evidence base

The `Makefile` exposes `make reproduce`, `make reproduce-check`, `make reproduce-full`, and task-specific targets such as `make precision-recall`, `make sound-fp`, `make diff-fuzz`, `make neg-fuzz`, and `make paper-evidence`. The orchestrator `reproducibility/reproduce_all.py` regenerates deterministic generated docs, tables, corpus manifests, benchmark artifacts, and the headline 60-bug figure before running the numeric-claim audit.

10.2 Numeric-claim audit

Every headline numeric claim in `README.md`, the conference paper, and the workshop paper is registered in `reproducibility/audit_numeric_claims.py`. For each claim, the harness checks that the number still appears in the cited source and recomputes it from the backing artifact. Claims can be `VERIFIED`, `MISMATCH`, `QUALIFIED_REGIME`, `QUALIFIED_ENV`, `ORPHAN`, or `SOURCE_MISSING`. The current committed audit has no uncovered README numeric tokens.

10.3 Artifact-evaluation capsule

The repository includes an artifact-evaluation package under `docs/artifact/` and a reproducibility capsule with pinned wheel locks, a Dockerfile, and `capsule/reproduce.sh`. The capsule verifies the live interpreter, regenerates deterministic artifacts, checks byte identity, and reruns the numeric audit. Badge evidence is mapped to in-tree paths for Available, Functional, Reusable, and Results Reproduced criteria.

10.4 Dashboard and release gates

`docs/dashboard/index.html` is a static site generated entirely from committed JSON artifacts. `evaluation/dashboard_baseline.json` freezes quality and integrity metrics so regressions are reviewable diffs rather than silent changes. `scripts/benchmark_delta.py` compares release snapshots with metric-direction awareness: recall must not drop, false positives must not rise, integrity populations must not shrink, and machine-dependent wall-clock metrics are reported without gating. Coverage gates hold selected public API and integration modules above 90% line coverage.

11 Deployment and governance

11.1 Packaging

TensorGuard is packaged as a typed Python project with `py.typed`, a pinned `z3-solver` dependency range, a CLI entrypoint, a pre-commit hook, and a pytest plugin. The roadmap includes PyPI, conda-forge, Docker, and pipx paths; the repository already contains the policy and harness pieces needed to make those releases reproducible.

11.2 Community benchmark and stubs

The leaderboard scores tools on the frozen real benchmark corpus by recomputing metrics from raw verdict files. External tools cannot self-report inflated numbers. The community stub registry gives framework and library authors a safe path to extend coverage: manifests must be declarative, carry provenance, and ship executed conformance cases. The governance docs make reviewability part of the technical contract.

11.3 Documentation and tutorials

The README intentionally surfaces limitations near the feature list. The Getting Started guide, "what it cannot do yet" document, Colab-style tutorials, examples gallery, artifact appendix, user-study protocol, PyTorch RFC draft, growth playbook, and dashboard are all tested for path existence or executable snippets where appropriate. Documentation is treated as a checked artifact, not post-hoc marketing prose.

12 Threats to validity and limitations

Not all PyTorch is in scope. PyTorch is too large for a single static checker to model completely. The verifiable fragment is explicit, and unsupported constructs produce UNKNOWN in sound mode. This means SAFE is meaningful, but UNKNOWN is a normal and expected answer on dynamic Python.

Some domains are diagnostic or partial. Phase facts are useful for train/eval diagnostics, but the current marginal contribution study classifies phase as diagnostic-only where it does not independently refute. The gradient domain catches `detach`-style forward-flow bugs but currently misses post-construction parameter freezes such as `requires_grad=False`; those misses are tagged in the artifact.

Generated corpora are not natural distributions. Generated and mutated corpora are valuable because they are redistributable and have executable labels, but they do not replace naturally-occurring bugs. The project therefore reports them alongside real benchmarks, GitHub-mined issue snapshots, and natural-distribution clean samples.

Some regeneration is environment-qualified. Lean, CUDA, and HuggingFace model downloads are not required for the standard CPU-only reproduction path. Claims depending on those environments are recorded with regeneration commands and classified as environment-qualified by the audit.

Formal proof scope is explicit. Lean proves many central rules and audits the public theorem corpus, but the entire Python implementation, parser, and every long-tail handler are not all inside a single theorem. The soundness-footprint artifacts state which verdicts traverse Lean-audited, pen-and-paper, tested-only, or uncovered handlers.

13 Related tools

Runtime shape libraries. Libraries such as jaxtyping and torchtyping check tensor contracts at runtime and often require annotations. They are useful when the annotated path is executed, but they do not statically prove unexecuted branches and do not cover device, phase, export, or distributed hazards by default.

Static tensor analyzers. PyTea is the closest academic baseline for static PyTorch shape checking. The repository builds and runs PyTea live where possible, reports a shape-only subcorpus where tools tie, and separately reports where TENSORGUARD's additional domains catch bugs PyTea is not designed to model.

PyTorch tracing and export. `torch.fx`, `torch.export`, TorchScript, and torch.compile can surface shape failures by tracing or compiling with concrete examples. They are powerful dynamic or capture-time tools, but they require instantiation and example inputs. TensorGuard uses these paths as frontends and baselines while retaining an input-free static path.

General static analyzers. Mypy, Pyright, and similar type checkers catch Python type errors but do not reason about tensor dimensions or cross-device placement. TensorGuard is complementary: it focuses on tensor metadata contracts rather than replacing Python type checking.

14 What makes the repository unusually strong

A typical research prototype has one impressive idea and a thin evaluation. TensorGuard has a different shape: its claim is that static tensor verification can be made operationally trustworthy. That requires several reinforcing layers:

1. a strict contract that refuses to overclaim;
2. transfer functions tagged by confidence;
3. real and generated corpora with executable labels;
4. fuzzing, mutation, dispatcher differential tests, and regression minimization;
5. formal proofs for the rules most likely to cause unsoundness;
6. claim audits so prose cannot drift from artifacts;
7. CI gates that make benchmark regressions reviewable;
8. adoption surfaces that integrate with how developers actually ship PyTorch code.

This combination is what makes the project a plausible candidate for upstream PyTorch influence: not that it already models every operator, but that it provides a disciplined path for growing coverage without sacrificing trust.

15 Next research and adoption frontier

The local roadmap now points beyond the completed campaign toward a best-paper-award and 1,000-star release, with one hundred concrete future work items numbered after the completed campaign. The most important next directions are not more marketing metrics; they are higher-value coverage and stronger evidence:

- deeper operator coverage for split/chunk, masked selection, nonzero, exact einsum diagonal semantics, and recurrent layers;
- deployment correctness for quantization, mixed precision, dynamic `torch.export` constraints, AOTInductor layouts, ONNX opsets, GGUF export, FSDP2, and pipeline-parallel stage boundaries;
- proof depth for einops, SDPA, chunk/split, distributions, named-tensor alignment, grid sampling, contiguity/stride algebra, and whole-module subject reduction over the expanded operator set;
- evaluation scale: 1,000+ provenance-rich real bugs, stratified recall and false-positive confidence intervals, registered held-out protocols, head-to-head baselines, and PR-history survival analysis;

- adoption: hosted demo, one-line GitHub Action and pre-commit setup, editor clients, Code Scanning trends, web explanations, Lightning/HF Trainer adapters, model-hub badges, a real-model gallery, and a public leaderboard refresh cadence.

16 Conclusion

TensorGuard turns tensor metadata checking from a runtime accident into an auditable static contract. It is already useful as a developer tool: it catches shape, device, dtype, phase, gradient, training-loop, distributed, quantization, export, einops, SDPA, MultiheadAttention, distribution, complex FFT/view, named-axis, vmap, sparse-layout, and sampler-grid hazards before execution; it reports precise diagnostics; it integrates with CI and code scanning; and it defaults to abstention rather than false certainty when the program leaves its fragment. It is also unusually well-supported as a research artifact: every headline number is tied to a regenerable artifact, every corpus has provenance, every known limitation is surfaced, and the formal proof story is executable enough to fail CI when it drifts.

A Evidence index

Table 4: Selected repository artifacts backing the paper.

Question	Artifact / command	Tests / gate
What programs are soundly verified?	SOUNDNESS_CONTRACT.md, src/soundness_contract.py	tests/test_soundness_contract.py, tests/test_soundness_mode.py
What source fragment is supported?	VERIFIABLE_FRAGMENT.md, src/verifiable_fragment.py	tests/test_verifiable_fragment_spec.py
Which operators are complete, sound, or heuristic?	operator_confidence_table.json, tensorguard operator-confidence	tests/test_operator_confidence.py
Does disabling a domain skip solver work?	reproducibility/check_flag_demo.json	tests/test_solver_domain_gating.py
Does CEGAR change real bug results?	tests/test_cegar_refined_contract.py, reproducibility/cegar_depth_ablation.md	tests/test_cegar_depth_ablation.py
How does TensorGuard compare to baselines?	evaluation/confusion_matrices.json, evaluation/significance.json	tests/test_precision_recall.py, tests/test_significance.py
Does sound mode false-alarm on clean models?	evaluation/sound_mode_fp.json	tests/test_sound_mode_fp.py
Does random clean fuzzing find false positives?	evaluation/diff_fuzz.json	tests/test_diff_fuzz.py
Does injected-fault fuzzing find false negatives?	evaluation/neg_fuzz.json	tests/test_neg_fuzz.py
Are disagreements minimized and frozen?	evaluation/minimize_demo.json, evaluation/triage_regressions.json	tests/test_minimize.py, tests/test_triage.py
Is there an extended redistributable corpus?	corpus_extended/manifest.json, reproducibility/corpus_extended_score.md	tests/test_corpus_extended.py
Is there a blind split?	corpus_extended/PRE_REGISTRATION.md, reproducibility/blind_split_eval.md	tests/test_blind_split.py
Does it agree with the live torch dispatcher?	reproducibility/differential_dispatcher.md	tests/test_differential_dispatcher.py
Does mutation testing catch genuine bugs?	reproducibility/mutation_clean_models.md	tests/test_mutation_clean_models.py
Does the localizer reduce effort?	evaluation/localization_effort.json	tests/test_localization_effort.py
How much operator coverage exists on real models?	evaluation/operator_frequency.json, evaluation/fx_trace_success.json	tests/test_operator_frequency.py, tests/test_fx_trace_success.py

Question	Artifact / command	Tests / gate
Are einops contracts faithful?	src/einops_verify.py, src/einops_source.py	tests/test_einops_verify.py, tests/test_einops_source.py
Is SDPA modeled faithfully?	src/sdpa_verify.py	tests/test_sdpa_verify.py
Is MultiheadAttention q/k/v and mask handling modeled faithfully?	src/mha_verify.py	tests/test_mha_verify.py
Are probabilistic batch/event shapes modeled faithfully?	src/distributions_verify.py	tests/test_distributions_verify.py
Are named-tensor refinements and alignments modeled faithfully?	src/named_tensor_verify.py	tests/test_named_tensor_verify.py
Is vmap batched-shape transfer modeled faithfully?	src/vmap_verify.py	tests/test_vmap_verify.py
Are sparse layout and block-size contracts modeled faithfully?	src/sparse_verify.py	tests/test_sparse_verify.py
Are geometric sampler grids modeled faithfully?	src/grid_sample_verify.py	tests/test_grid_sample_verify.py
Are training-loop hazards checked?	reproducibility/training_loop_hazards.md	tests/test_training_loop_checks.py
Are tensor-parallel configs checked?	reproducibility/tensor_parallel_sharding.md	tests/test_tensor_parallel_checks.py
Are quant/export hazards checked?	reproducibility/quant_export_safety.md	tests/test_quant_export_checks.py
Is the Lean library sorry-free?	lean/TensorGuard/, tests/test_lean_whole_pipeline_audit.py	lake build TensorGuard in slow environments; axiom audit tests
Are headline numbers audited?	reproducibility/numeric_claims_audit.json	tests/test_numeric_claims_audit.py
Can the evidence be reproduced?	make reproduce, make reproduce-check, capsule/reproduce.sh	tests/test_reproduce_harness.py, tests/test_capsule_manifest.py

B Experiment catalogue

The repository deploys the following benchmark and validation families:

1. **Ground-truth real benchmarks:** content-addressed clean and buggy nn.Modules with provenance and eager-runtime validation.
2. **GitHub mining:** frozen public issue/PR snapshot from verbatim PyTorch runtime signatures.
3. **Precision/recall baselines:** TensorGuard, PyTea, runtime forward, runtime backward, and no-op on the same corpus.

4. **Statistical rigor:** McNemar tests, bootstrap intervals, effect sizes, multiple-comparison correction, and power analysis.

5. **False-positive hunts:** clean generated templates, random architecture fuzzing, false-positive stress grids, and natural-distribution clean models.

6. **False-negative hunts:** injected genuine faults, mutation testing, latent-bug corpora, and full dispatcher differential tests.

7. **Frontend studies:** fx trace success, export/fx reconciliation, cross-version stability, cross-Python determinism, and safe blocked-import checks.

8. **Operator studies:** registry coverage, frequency-weighted real model coverage, conformance oracle checks, shape algebra properties, einops fuzzing, SDPA fuzzing, and probabilistic distribution-shape fuzzing.

9. **Deployment checks:** training-loop hazards, tensor-parallel sharding, quantization/export safety, ONNX/export package gates, and upstream hook demos.

10. **Reproducibility checks:** numeric claims audit, artifact badges, capsule manifest, evidence dashboard, benchmark-delta gate, and one-command paper evidence generation.

C Capability matrix

This matrix is intentionally implementation-facing: it records what a user, reviewer, or future contributor can find in the repository today and which part of the system owns the behavior.

Table 5: Major shipped capabilities grouped by surface area.

Surface	**Capability**	**Primary implementation or evidence**
Core verifier	Zero-annotation `nn.Module` architecture verification with shape, dtype, device, phase, gradient, stride, and layout facts	`src/api.py`, `src/smt/solver.py`
Core verifier	Three-valued verdicts SAFE/UNKNOWN/UNSAFE with `sound`, `balanced`, and `heuristic` modes	`src/soundness_contract.py`, `tests/test_soundness_mode.py`
Core verifier	Solver-domain gates for device, phase, and gradient checks that remove constraints before Z3, not after the verdict	`tests/test_solver_domain_gating.py`, `reproducibility/check_flag_demo.json`
Core verifier	CEGAR-discovered infeasible contracts promoted to real bugs	`tests/test_cegar_refined_contract.py`, `reproducibility/cegar_depth_ablation.md`
Operator model	Confidence-tagged transfer registry and CLI inspection	`src/operator_confidence.py`, `operator_confidence_table.json`

Surface	Capability	Primary implementation or evidence
Operator model	Precise reshape/view/flatten, broadcasting, einsum, convolution, normalization, dtype-promotion, and device-transfer semantics	`tests/test_shape_algebra_properties.py`, `tests/test_lean_conformance.py`
Library model	einops `rearrange`/`reduce`/`repeat` checker with source-level bug reports	`src/einops_verify.py`, `src/einops_source.py`
Library model	SDPA shape and mask checker for attention hot paths	`src/sdpa_verify.py`, `tests/test_sdpa_verify.py`
Library model	MultiheadAttention packed/unpacked q/k/v, mask, key-padding, and weight-output contracts	`src/mha_verify.py`, `tests/test_mha_verify.py`
Library model	`torch.distributions` batch/event/log-prob shape checker	`src/distributions_verify.py`, `tests/test_distributions_verify.py`
Library model	PyTorch named-tensor `refine_names`/`align_to` checker with source-level bug reports	`src/named_tensor_verify.py`, `tests/test_named_tensor_verify.py`
Library model	`torch.vmap`/functorch mapped-axis removal and `out_dims` insertion	`src/vmap_verify.py`, `tests/test_vmap_verify.py`
Library model	`torch.sparse` COO/CSR/CSC/BSR/BSC layout and blocksize checks	`src/sparse_verify.py`, `tests/test_sparse_verify.py`
Library model	`grid_sample`/`affine_grid` sampler-rank, batch, dtype, and output-grid contracts	`src/grid_sample_verify.py`, `tests/test_grid_sample_verify.py`
Frontends	Safe AST source parser for untrusted model files	`src/safe_loader.py`, `tests/test_security.py`
Frontends	`torch.fx` trace lowering across real torchvision models	`evaluation/fx_trace_success.json`
Frontends	`torch.export`/Dynamo reconciliation with fx	`evaluation/frontend_reconciliation.json`
Frontends	Flax/JAX frontend lowering into the shared IR	`src/flax_extractor.py`, `tests/test_flax_frontend.py`
Extensibility	Governed declarative community stubs with provenance and executed conformance cases	`community_stubs/`, `src/stub_governance.py`
Training	Static checks for detached loss, missing or misordered `zero_grad`, missing `step`, and unsafe fp16 autocast	`src/training_loop_checks.py`, `reproducibility/training_loop_hazards.md`
Distributed	Tensor-parallel shard divisibility and communication-flag checks	`src/tensor_parallel_checks.py`, `reproducibility/tensor_parallel_sharding.md`

Surface	Capability	Primary implementation or evidence
Deployment	Quantization, export, ONNX, and package-gate safety checks	src/quant_export_checks.py, reproducibility/quant_export_safety.md
Developer UX	Human diagnostics, JSON, JUnit, SARIF, and GitHub annotation reporters	src/reporters.py, tests/test_reporters.py
Developer UX	Pre-commit hook, pytest plugin, GitHub Action path, and baseline/-suppression support	src/precommit.py, src/pytest_tensorguard.py
Developer UX	Upstream-style register_forward_pre_hook and @verifiable decorator reference implementation	src/upstream_hook.py, reproducibility/upstream_hook_demo.md
Documentation	Generated soundness and fragment specs plus concise onboarding	SOUNDNESS_CONTRACT.md, VERIFIABLE_FRAGMENT.md, GETTING_STARTED.md, LIMITATIONS.md
Formal methods	Lean rules for core operators, reduced products, CEGAR, fragment modes, and SMT encoding faithfulness	lean/TensorGuard/, tests/test_lean_whole_pipeline_audit.py
Benchmarks	Frozen real benchmark corpus with datasheet and citation metadata	real_benchmarks/manifest.json, real_benchmarks/DATASHEET.md
Benchmarks	Extended generated corpus, held-out blind split, false-positive stress set, natural-distribution study, and mutation corpus	corpus_extended/, reproducibility/blind_split_eval.md
Benchmarks	Public GitHub-mined bug snapshot and offline issue-miner proposal flow	experiments_v5/github_bug_mining/, corpus_extended/issue_miner.py
Evidence	Numeric-claim audit, dashboard, artifact capsule, benchmark delta gate, and paper-evidence target	reproducibility/audit_numeric_claims.py, docs/dashboard/index.html, scripts/benchmark_delta.py

D Selected numerical claims

Claim	Backing artifact
117 operator confidence rows: 75 complete, 36 sound, 6 heuristic	`operator_confidence_table.json`
16 frozen real benchmarks: 8 clean, 8 buggy	`real_benchmarks/manifest.json`
2,704 GitHub-mined public bugs: 2,394 shape, 310 device	`experiments_v5/github_bug_mining/mined_bugs_manifest.json`
TP=8, FP=0, TN=8, FN=0 on frozen real benchmarks	`evaluation/confusion_matrices.json`
0 false positives on 80 sound-mode clean models	`evaluation/sound_mode_fp.json`
0 false positives on 200 random clean fuzzed models	`evaluation/diff_fuzz.json`
281/281 genuine injected faults caught	`evaluation/neg_fuzz.json`
6/8 latent bugs caught vs. 0/8 for the strongest dynamic baseline	`evaluation/hard_recall.json`
50 minimal bug reproducers frozen across nine mechanisms	`evaluation/triage_regressions.json`
94.86% frequency-weighted operator coverage on 13 torchvision models	`evaluation/operator_frequency.json`
21/21 torchvision models traced and lowered by fx	`evaluation/fx_trace_success.json`
371 public Lean theorems discovered and axiom-audited	`tests/test_lean_whole_pipeline_audit.py`
12 verified, one regime-qualified, one environment-qualified numeric claims	`reproducibility/numeric_claims_audit.json`

E Manual bibliography

References

[1] PyTorch Contributors. PyTorch: An imperative style, high-performance deep learning library.

[2] Leonardo de Moura and Nikolaj Bjorner. Z3: An efficient SMT solver.

[3] The cvc5 Developers. cvc5: A versatile and industrial-strength SMT solver.

[4] PyTea Contributors. PyTea: Static analysis for tensor shape errors in deep learning programs.

[5] Lean Prover Community. The Lean 4 theorem prover and programming language.

[6] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement.

[7] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages.